



1

Objectives

After completing this module, you will be able to:

- OBJECTIVE 01**
Describe the AMD Vivado™ logic analyzer (VLA) and its fundamental components
- OBJECTIVE 02**
Enumerate the benefits of the Vivado logic analyzer
- OBJECTIVE 03**
Describe the basic probing flows to debug a design

The illustration depicts a stylized human head in profile, facing left. Inside the head, there are circuit board patterns and a glowing yellow circle. Below the head, there are several small figures of people interacting with electronic equipment. One person is sitting on the ground with a laptop, another is standing and looking at a large circular device, and a third is standing next to a rack of equipment. The entire scene is rendered in a clean, modern, light blue and orange color palette.

© Copyright 2023 Advanced Micro Devices, Inc.

AMD together we advance_

2

Logic Analyzer Basics

Designers spend more than 40% of their time in debugging

Vivado™ logic analyzer debug solution reduces the time required for verification and debugging



3

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance...

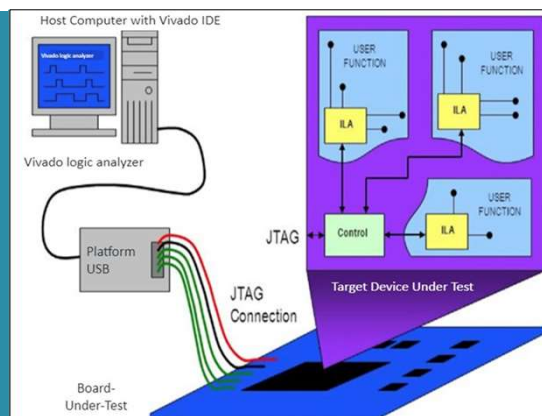
3

Logic Analyzer Basics

Designers can maximize productivity with a solid understanding of the underlying function of this Vivado™ debug tool

Accelerate your success of design by:

- Reducing risk of issues
- Rapidly locating problems
- Fixing the bug quickly



4

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance...

4

Motivation for the Vivado Logic Analyzer Tool

- 01 → Functionality of a conventional logic analyzer can be built into any AMD FPGA
- 02 → The triggering and data collection capability of a logic analyzer can be implemented using the Vivado™ debug cores
- 03 → Vivado debug cores do not consume any I/O pins
- 04 → VLA allows different triggering scenarios
- 05 → Single JTAG connection to the PC can be used for programming

5

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

5

Vivado Logic Analyzer



What is the Vivado™ logic analyzer?

Vivado logic analyzer is integrated into the Vivado Design Suite

6

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

6

Vivado Logic Analyzer

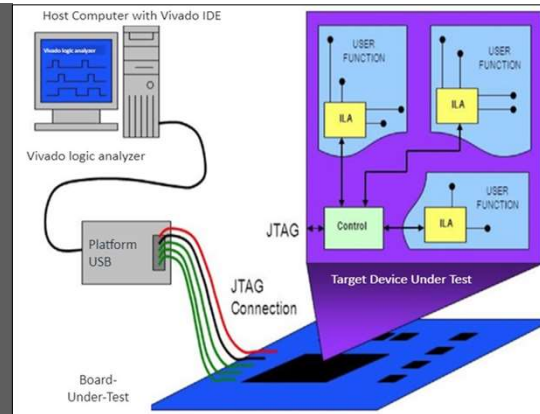
Powerful yet easy-to-use debug tool

Synergistic role with simulation

Vivado™ logic analyzer can be used in three contexts:

- Verification
- Debugging
- Data capture

Interact with debug IP cores



7

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

7

Vivado Logic Analyzer Tool

The screenshot shows the Vivado Logic Analyzer Tool interface. The Hardware Manager on the left shows the hardware tree with 'hw_ila_1' selected. The Properties window shows details for 'hw_ila_1'. The main window displays the Dashboard with core status, capture status, trigger setup, and a waveform window showing data capture.

Vivado™ Logic Analyzer Tool

8

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

8

VLA Comprised of...

Comprised of two basic items:

- Debug cores
 - ILA, System ILA, VIO, JTAG to AXI Master, and IBERT, etc.
- Debug flows
 - HDL instantiation and netlist insertion flows

9

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance...

9

Benefits of Logic Debug

Vivado™ logic
analyzer captures
data within an FPGA

1

Debug Probes window
shows nets probed for
debugging

3

Allows flexible
targeted probing of
HDL design using the
MARK_DEBUG
property

2

Sets most debug
parameters
during run time

4

10

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance...

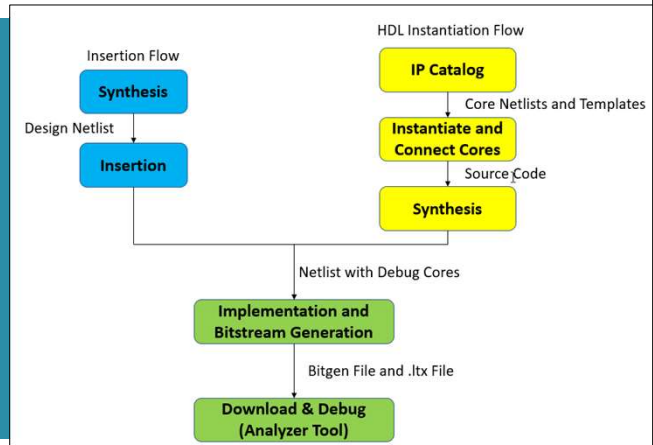
10

Vivado Logic Analyzer Probing Flows

Two Vivado™ logic analyzer probing flows:

- HDL instantiation flow
- Netlist insertion flow

HDL instantiation flow and netlist insertion flows can be mixed



11

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance...

11

HDL Instantiation or Insertion Flow



Debug flows: a comparison

HDL Instantiation Flow

Allows user to connect to any signal

User must modify HDL code to instantiate cores

Netlist Insertion Flow

Recommended flow: easy to add/remove debug core

Vivado™ tool auto-inserts debug core into post-synthesis netlist

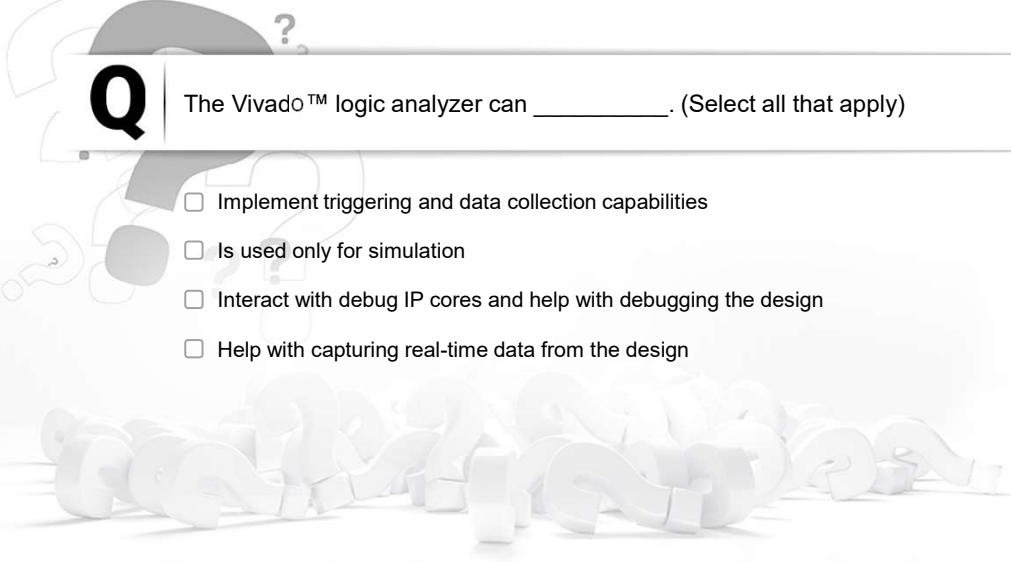
12

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance...

12

Apply Your Knowledge

**Q**

The Vivado™ logic analyzer can _____. (Select all that apply)

- ☐ Implement triggering and data collection capabilities
- ☐ Is used only for simulation
- ☐ Interact with debug IP cores and help with debugging the design
- ☐ Help with capturing real-time data from the design

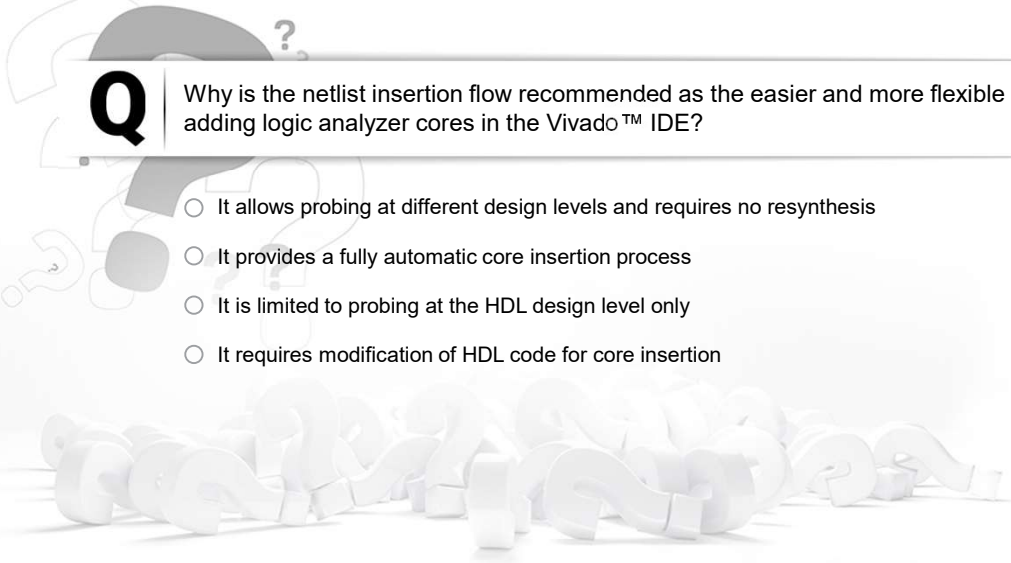
13

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

13

Apply Your Knowledge

**Q**

Why is the netlist insertion flow recommended as the easier and more flexible option for adding logic analyzer cores in the Vivado™ IDE?

- ☐ It allows probing at different design levels and requires no resynthesis
- ☐ It provides a fully automatic core insertion process
- ☐ It is limited to probing at the HDL design level only
- ☐ It requires modification of HDL code for core insertion

14

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

14

Summary

01 The Vivado™ logic analyzer is extremely effective in the verification and debugging of FPGA designs

02 Debug cores can implement triggering and data collection capabilities of the logic analyzer

03 The logic analyzer is comprised of two main items:

- Debug cores
- Flows for generating and inserting debug cores

04 Insertion and HDL instantiation flows are the Vivado IDE debug probing flows

15

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance_

15

Introduction to Triggering

2023.2

AMD
together we advance_

16

Objectives

After completing this module, you will be able to:

OBJECTIVE 01

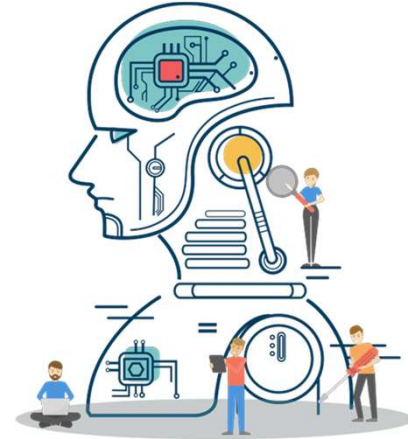
Describe the trigger mechanism of the AMD Vivado™ logic analyzer

OBJECTIVE 02

Explain how to use the Run Trigger option

OBJECTIVE 03

Display captured data using the waveform view



17

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

17

Triggering

Triggering is...

- Capturing data when a condition is met
- Condition to set the trigger can be anything
- Data can be displayed either prior to or after the occurrence of the trigger or both



18

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

18

Triggering

- 1 Plays a key role in debugging
- 2 Enables the ILA cores to capture data at the proper time
- 3 Helps in finding a specific event and avoids the uninteresting data
- 4 Two or more different ILAs can be used

19

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

19

Setting Up Triggering

Once settings are selected, hardware is generated and triggering is fixed

Tailor trigger to specific need by combining core capabilities

Configuring the ILA cores:

- HDL instantiation flow
- Netlist insertion flow

Analyzer tool with the Debug Probes window

20

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

20

ILA Dashboard

The screenshot displays the ILA Dashboard for a hardware instance named `hw_ila_1`. The interface is organized into four main panels:

- Settings - hw_ila_1:** Contains configuration options for the ILA.
 - Trigger Mode Settings:** Trigger mode is set to `BASIC_ONLY`.
 - Capture Mode Settings:** Capture mode is `ALWAYS`, Number of windows is `1`, Window data depth is `32768`, and Trigger position in window is `8192`.
 - General Settings:** Refresh rate is set to `500` ms.
- Status - hw_ila_1:** Shows the current state of the ILA.
 - Core status:** Indicators for Idle, Pre-Trigger, Waiting for Trigger, Post-Trigger, and Full.
 - Capture status:** Window 1 of 1, Window sample 0 of 32768, Total sample 0 of 32768.
 - Trigger Setup:** A table showing the trigger condition: `rx_data_rdy == [B] R`.
- Waveform - hw_ila_1:** Displays a digital signal trace for the trigger condition `rx_data_rdy`.

21

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance.

21

ILA Dashboard – Settings

This screenshot provides a detailed view of the ILA configuration settings for `hw_ila_1`. The settings are organized into four main sections:

- Trigger Mode Settings:** Trigger mode is set to `BASIC_ONLY`.
- Capture Mode Settings:** Capture mode is `ALWAYS`, Number of windows is `1`, Window data depth is `32768`, and Trigger position in window is `8192`.
- General Settings:** Refresh rate is set to `500` ms.
- Status - hw_ila_1:** Shows the current state of the ILA.
 - Core status:** Indicators for Idle, Pre-Trigger, and Waiting for Trigger.
 - Capture status:** Window 1 of 1, Window sample 0 of 32768, Total sample 0 of 32768.
 - Trigger Setup:** A table showing the trigger condition: `rx_data_rdy == [B] R`.

22

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance.

22

ILA Dashboard – Status

Settings - hw_ila_1

Trigger Mode Settings

Trigger mode: BASIC_ONLY

Capture Mode Settings

Capture mode: ALWAYS

Number of windows: 1 [1 - 32768]

Window data depth: 32768 [1 - 32768]

Trigger position in window: 8192 [0 - 32767]

General Settings

Refresh rate: 500 ms

Status - hw_ila_1

Core status

Idle Pre-Trigger Waiting for Trigger Post-Trigger Full

Capture status

Window 1 of 1 Window sample 0 of 32768 Total sample 0 of 32768

Idle Idle Idle

Trigger Setup - hw_ila_1

Name	Operator	Radix	Value	Port
rx_data_rdy	==	[B]	R	probe...

23

ILA Dashboard – Trigger Setup

Status - hw_ila_1

Core status

Idle Pre-Trigger Waiting for Trigger Post-Trigger Full

Capture status

Window 1 of 1 Window sample 0 of 32768 Total sample 0 of 32768

Idle Idle Idle

Trigger Setup - hw_ila_1

Name	Operator	Radix	Value	Port
rx_data_rdy	==	[B]	R	probe...

24

ILA Dashboard – Capture Setup

Name	Radix	Value
GPIO_BUTTONS_dly[1:0]	[E]	ONE

25

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

25

ILA Dashboard – Waveform

Name	Value
> rx_data[7:0]	41
uart_rx_i...d_i0_n_0	1
rx_data_rdy	0

26

© Copyright 2023 Advanced Micro Devices, Inc.

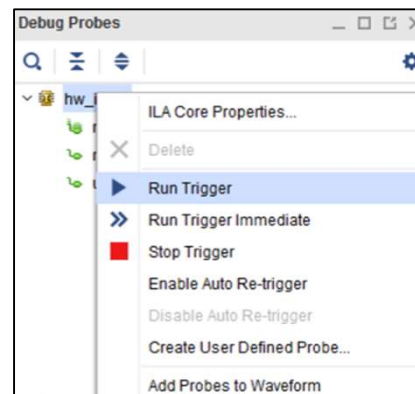
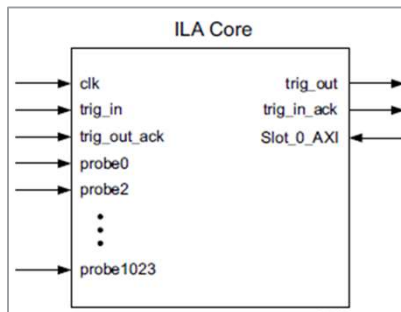
AMD
together we advance.

26

ILA Terminology

- 1024 unique probes
- Each of width ranging from 1 to 4096
- ILA specifies the match type and sensitivity

Once the trigger condition is met, the ILA core displays the waveform



27

© Copyright 2023 Advanced Micro Devices, Inc.

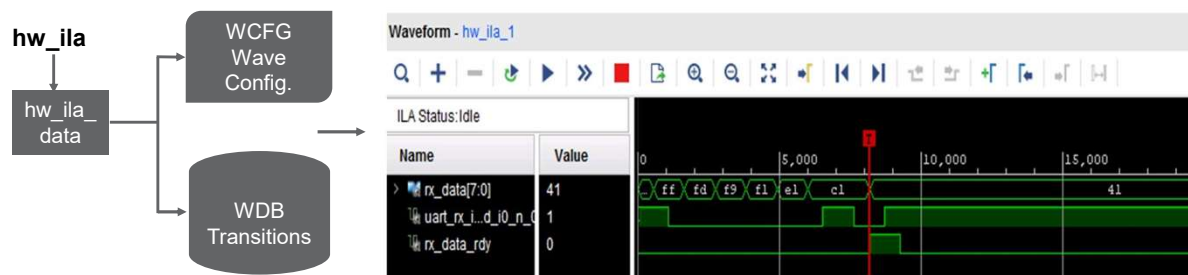
AMD
 together we advance...

27

Displaying Captured Data

ILA waveform viewer provides a powerful way to analyze data

After successfully triggering, the Vivado™ IDE automatically populates a corresponding waveform viewer



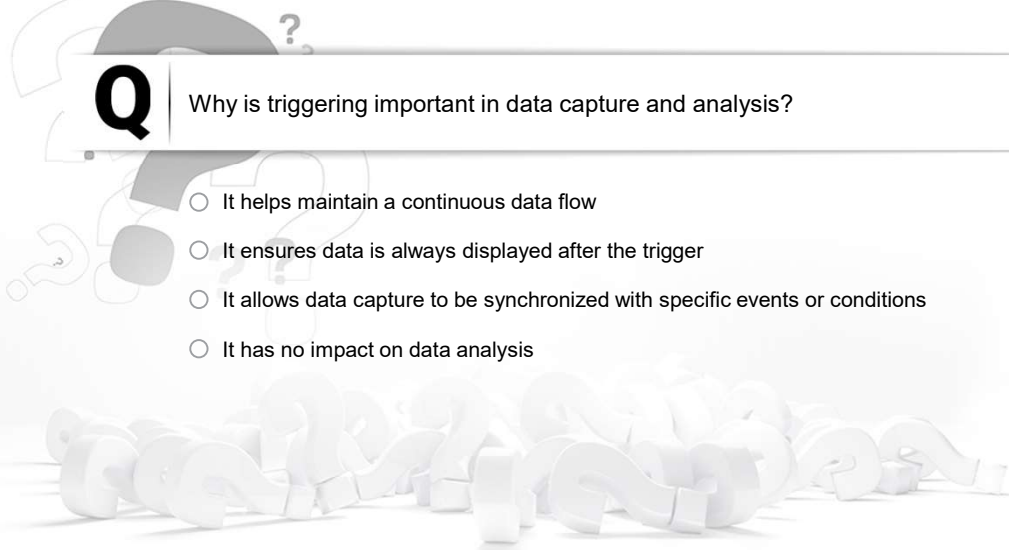
28

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

28

Apply Your Knowledge



Q

Why is triggering important in data capture and analysis?

- ☐ It helps maintain a continuous data flow
- ☐ It ensures data is always displayed after the trigger
- ☐ It allows data capture to be synchronized with specific events or conditions
- ☐ It has no impact on data analysis

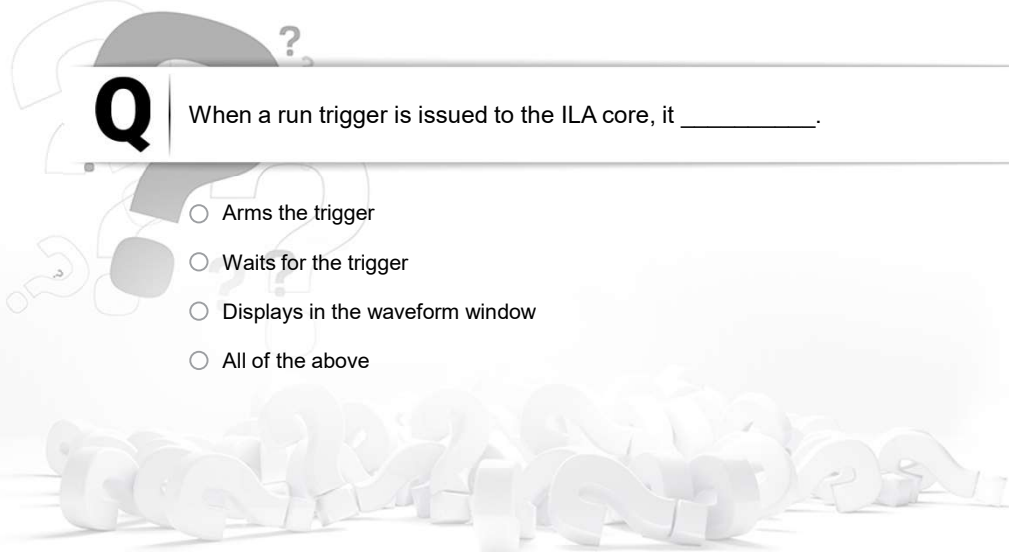
29

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

29

Apply Your Knowledge



Q

When a run trigger is issued to the ILA core, it _____.

- ☐ Arms the trigger
- ☐ Waits for the trigger
- ☐ Displays in the waveform window
- ☐ All of the above

30

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

30

Summary

01 The ILA core can trigger on many events with flexible event combinations

02 The Vivado™ logic analyzer leverages the ILA core's sampling capability

03 Triggering

- Enables tailoring for specific sensitivity
- Supports the combining of triggering conditions

04 Data can be displayed as a waveform

- Each ILA needs a separate waveform window

31

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance_

31

Debug Cores

2023.2

AMD
together we advance_

32

Objectives

After completing this module, you will be able to:

OBJECTIVE 01

Describe the ILA core and its properties

OBJECTIVE 02

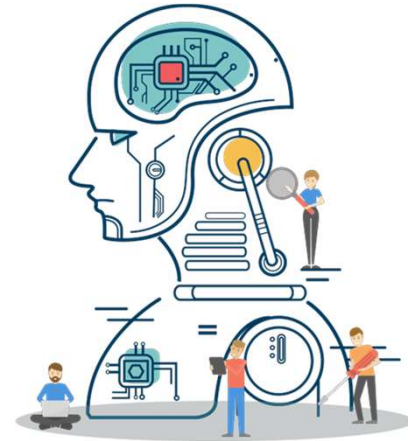
Describe VIO and IBERT core usage

OBJECTIVE 03

Describe what the debug core hub is and why it is used

OBJECTIVE 04

Describe debug core hub insertion and customization



33

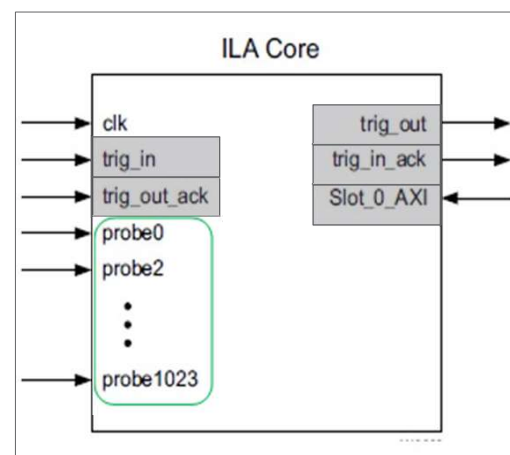
© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance_

33

Integrated Logic Analyzer (ILA)

- Used to monitor any design
- Signals are connected to ILA core clock and probe inputs
- Signals sampled at design speed and stored in block RAM
- User configurable up to 1024 probe ports
- Up to 4096 signals per probe port
- Monitor types: Native and AXI
- Communication with the ILA core is conducted using an auto-instantiated debug core hub



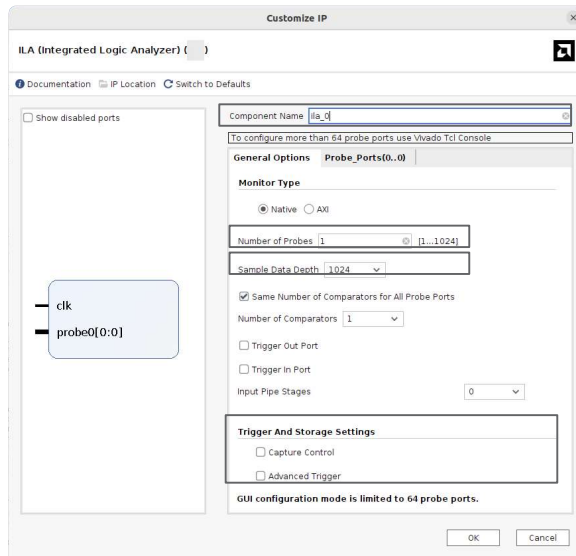
34

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance_

34

ILA Core Wizard

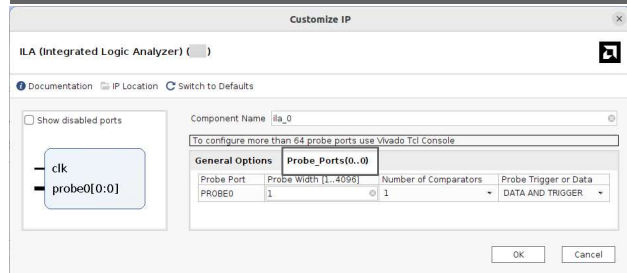


User-selectable parameters are:

- Component name
- Number of probe ports
- Sample data depth
- Trigger and storage settings
- Width for each probe input

ILA debug core supports:

- HDL instantiation flow
- Netlist insertion flow



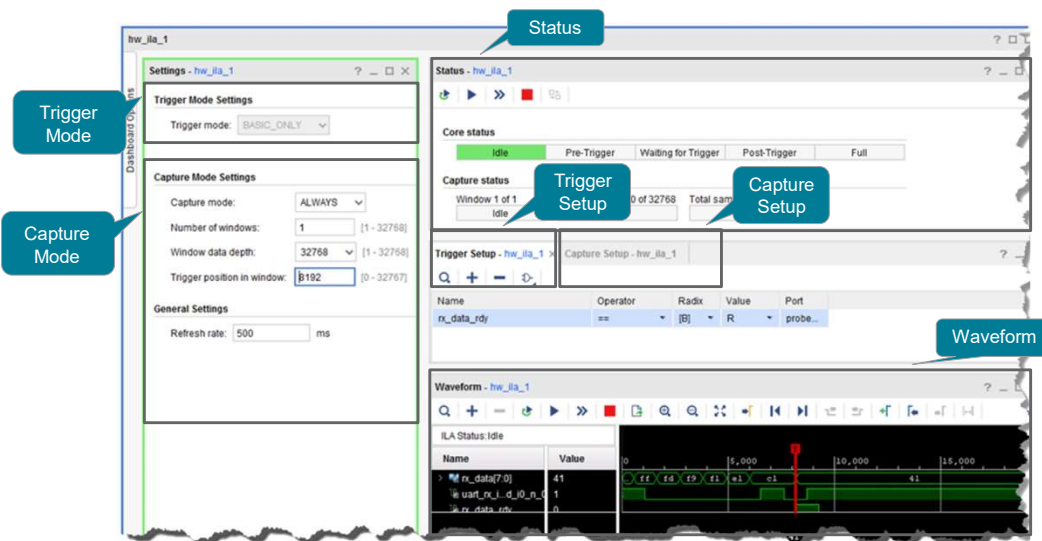
35

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

35

ILA Dashboard



36

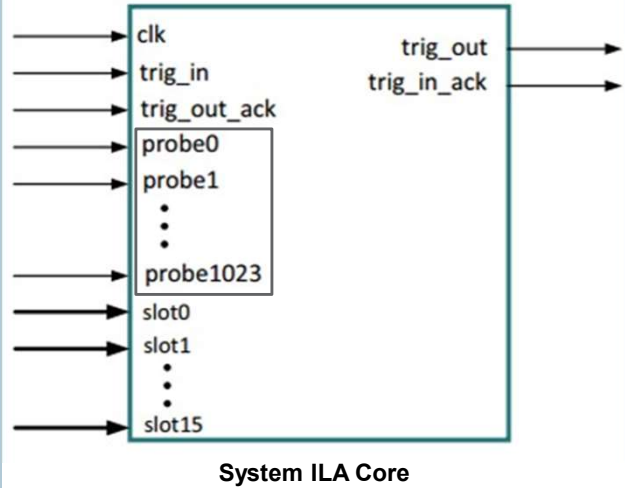
© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

36

System ILA Core

- Used to monitor the internal signals and interfaces
- Allows in-system debugging
- Signals and interfaces in the FPGA design are connected to the System ILA probe and slot inputs
- Sampled at design speeds and stored using on-chip block RAM
- User configurable up to 1024 probe ports
- Up to 4096 signals per probe port



37

© Copyright 2023 Advanced Micro Devices, Inc.

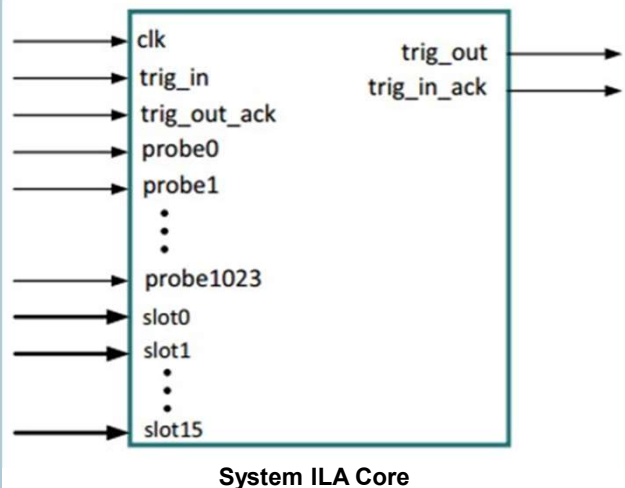
AMD
 together we advance...

37

System ILA Core

Core parameters specify:

- Number of probes and interface slots
- Interface types, trace sample depth
- Data and trigger properties of probes and interfaces
- Number of comparators and the width for each probe



38

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

38

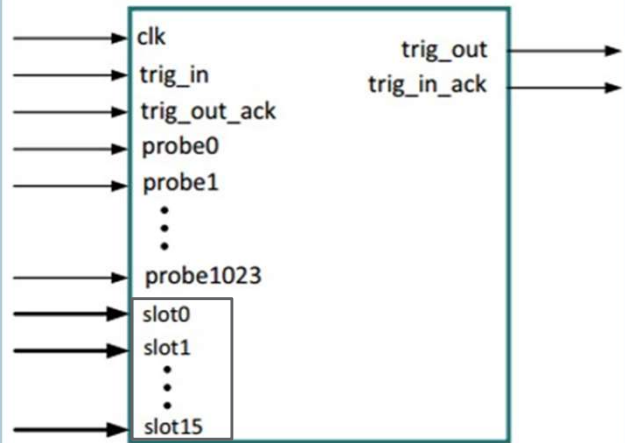
System ILA Core

Monitor types:

- Native
- Interface

Slot_0_AXI → AXI interface

Up to 16 interface slots



System ILA Core

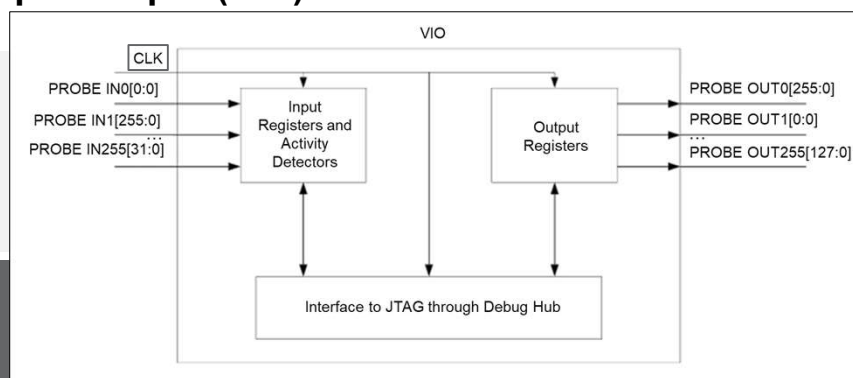
39

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

39

Virtual Input/Output (VIO)



Customizable core

No on-chip or off-chip RAM is required

Presents and captures data on the design clock

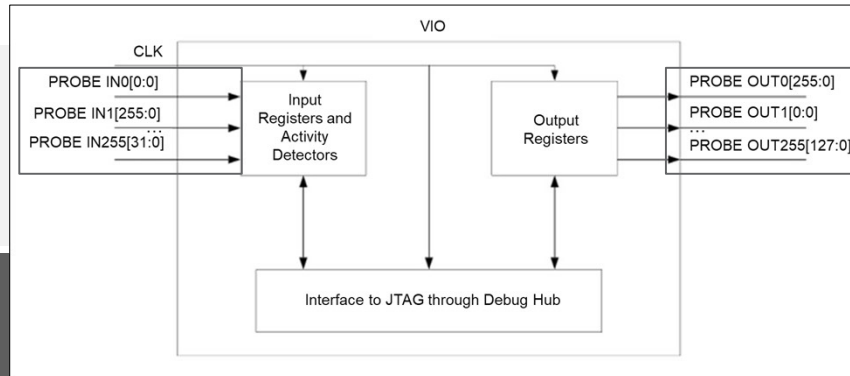
40

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

40

Virtual Input/Output (VIO)



Insert “virtual” pins into the design
Can produce up to 65 input and output probes
Each port can be 256 bits wide

Inputs and outputs can be visualized in different ways
Cores can be used as virtual LEDs, pushbutton, or toggle switches

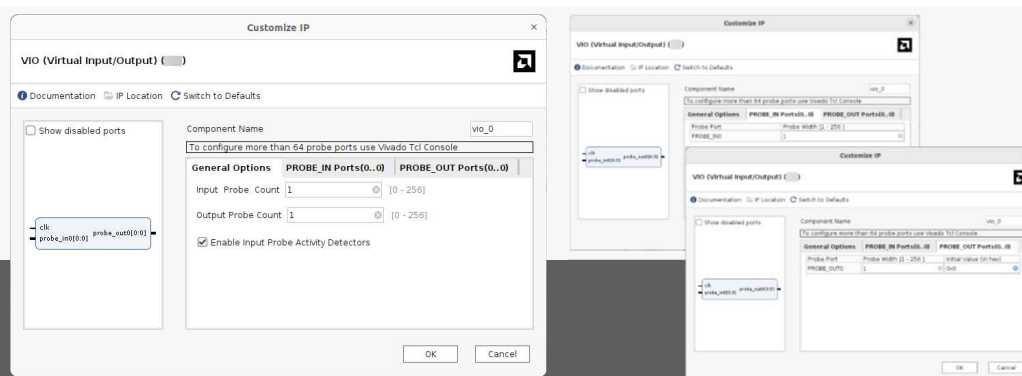
41

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

41

VIO Core Wizard



User-selectable parameters:

- Component name
- Number of input and output probe ports
- Width of each probe port

This debug core supports only the HDL instantiation flow

42

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

42

Virtual Input/Output (VIO)

VIO Ports and Parameters

VIO Dashboard

Signal Name	I/O	Description
CLK	I	Design clock used to register input ports and output ports. This is required
PROBE_IN<m> [<n> - 1:0]	I	Input probe port number <m> (where <m> can be 0 to 256) of width <n> (where <n> can be 1 to 256). For a 1-bit wide port, use PROBE_IN<m>[0:0]
PROBE_OUT<m> [<n> - 1:0]	O	Output probe port number <m> (where <m> can be 0 to 256) of width <n> (where <n> can be 1 to 256). For a 1-bit wide port, use PROBE_OUT<m>[0:0]

43

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance.

43

Virtual Input/Output (VIO)

VIO Ports and Parameters

VIO Dashboard

- Shows all status and control information pertaining to a given VIO core
- Drag-and-drop VIO probes from the Debug Probes window
- Monitor “virtual” pins in your design
- Inputs appear as LEDs or text
- Outputs appear as toggle buttons

Name	Value	Acti...	Directi...	VIO
rx_data[7:0]	[H] 6C		Input	hw_vio_1
rx_data[7]			Input	hw_vio_1
rx_data[6]			Input	hw_vio_1
rx_data[5]			Input	hw_vio_1
rx_data[4]			Input	hw_vio_1
rx_data[3]			Input	hw_vio_1
rx_data[2]			Input	hw_vio_1
rx_data[1]			Input	hw_vio_1
rx_data[0]			Input	hw_vio_1
virtual_button	1		Output	hw_vio_1

44

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance.

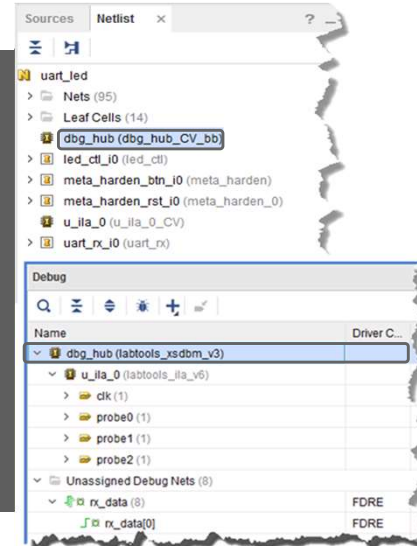
44

Debug Core Hub

- Interfaces between JTAG BSCAN and Vivado™ Design Suite debug cores
- Only one dbg_core_hub is required in a design
- Debug cores are connected to provide connectivity
- Designer can view and change BSCAN user scan chain
- Designer must ensure that the clock going to the dbg_hub is a free-running clock
- `connect_debug_port` Tcl command is used to connect the clk pin

Debug Core Hub
Insertion

Debug Core Hub
Customization

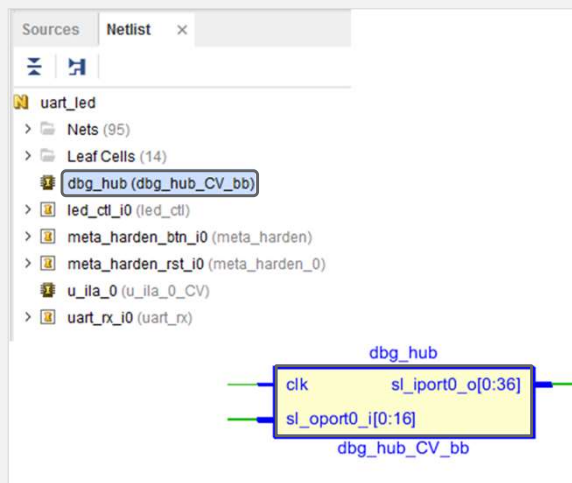


AMD
together we advance.

45

Debug Core Hub Insertion

- Debug cores are inserted into the netlist initially as black boxes (not implemented yet)
- Instantiated at the top level of the design hierarchy
- Netlist and Schematic views display new cores
- Black-box module **dbg_hub** displayed in Netlist view

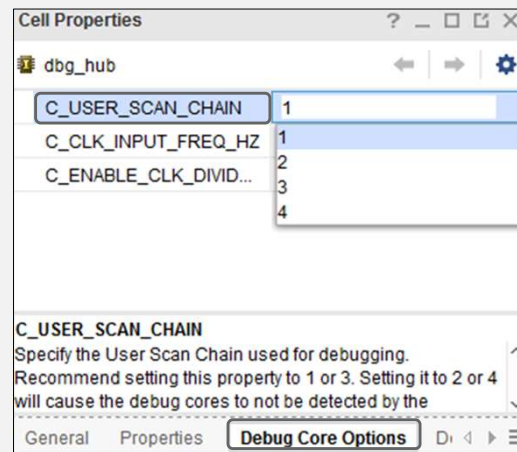


AMD
together we advance.

46

Debug Core Hub Customization

- To view or change the BSCAN user chain index of dbg_hub: Select **dbg_hub** to open the Cell Properties window and click **Debug Core Options**
- View or change the **C_USER_SCAN_CHAIN** property



47

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

47

Debug Bridge

The Debug Bridge IP core is...

Provides multiple communication options with debug cores

Enables remote debugging over a Xilinx Virtual Cable (XVC) server

Enables tandem debug with field update designs

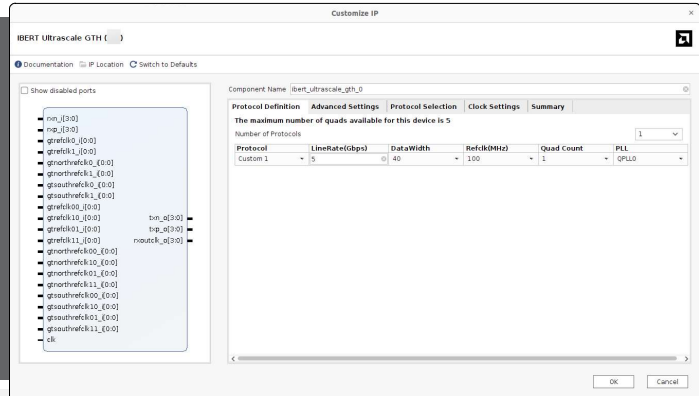
48

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

48

- Debugs, verifies, and optimizes high-speed serial communication channels
- Provides access to the multi-gigabit transceivers
- Performs bit-error ratio analysis on channels composed of MGT/GTP/GTX/GTH components
- Analyzer tool provides user-adjustable parameters to test various configurations



AMD
together we advance...

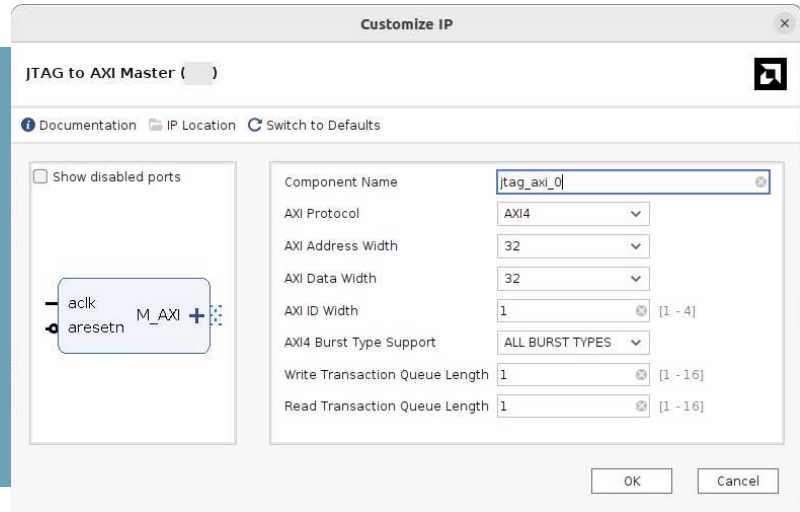
- Use the IBERT Serial Analyzer design to address a range of in-system debug and validation problems—from simple clocking and connectivity issues to complex margin analysis and channel optimization issues
- Use the IBERT Serial Analyzer to measure signal quality after receiver equalization

JTAG to AXI Master

Used to generate AXI transactions

Support for 32- and 64-bit wide AXI and AXI-Lite interfaces

Recommendation: Use this core to generate AXI transactions and debug/drive AXI signals internal to an FPGA at run time



51

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance.

51

Apply Your Knowledge

Q

What is the purpose of the VIO core?

- ☐ To monitor and drive internal FPGA signals in real time
- ☐ To sample and store signals using on-chip block RAM
- ☐ To provide advanced trigger equations for debugging
- ☐ To connect external devices to the FPGA via AXI interface

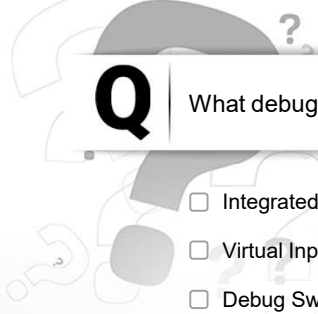
52

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance.

52

Apply Your Knowledge



Q

What debug cores are supported in the Vivado™ IP catalog? (Select all that apply)

- ☐ Integrated Logic Analyzer
- ☐ Virtual Input/Output
- ☐ Debug Switch
- ☐ AXI to JTAG Master
- ☐ IBERT

53

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

53

Summary

- 01** ILA debug core is used to monitor the design
- 02** VIO debug core inserts “virtual” pins into the design
- 03** Debug hub core is used to connect debug cores to the JTAG scan chain
- 04** Debug bridge provides connectivity to debug hub and debug cores
- 05** IBERT debugs, verifies, and optimizes high-speed serial communication channels

54

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

54

Netlist Insertion Debug Probing Flow

2023.2

AMD
together we advance_

55

Objectives

After completing this module, you will be able to:

OBJECTIVE 01

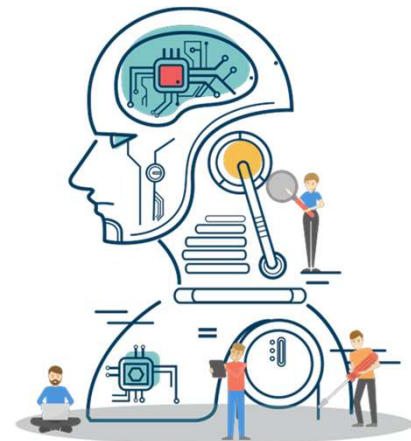
Describe the process of inserting the ILA core into a synthesized design netlist in the Vivado™ IDE

OBJECTIVE 02

Identify the strengths and weaknesses of the netlist insertion flow

OBJECTIVE 03

Use Tcl commands to insert debug cores and analyze the design



56

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance_

56

Introduction to the Netlist Insertion Flow

The Vivado™ IDE enables in-system debugging using the Vivado logic analyzer (VLA) through two flows:

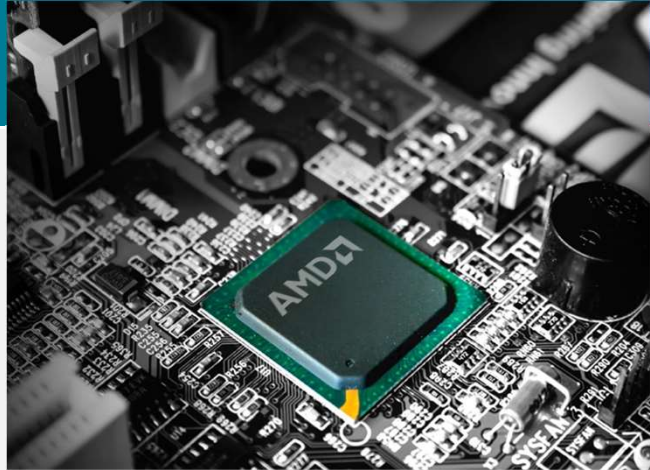


HDL instantiation flow



Netlist insertion flow

In the netlist insertion flow, the debug core is inserted into the synthesized netlist of a design



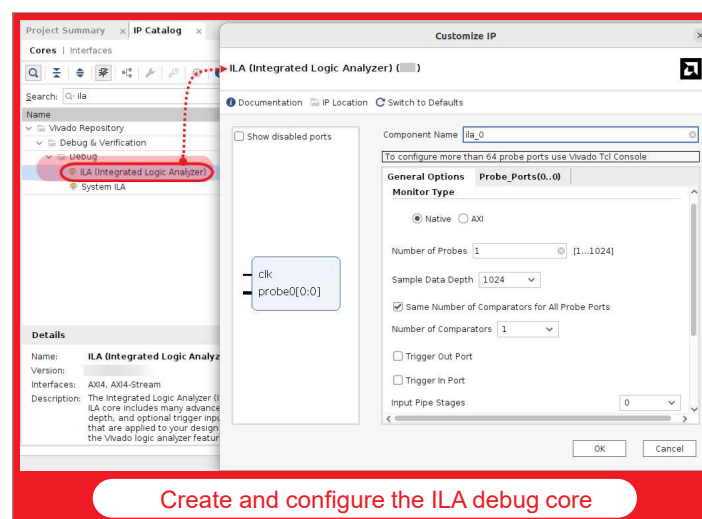
57

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

57

Introduction to the Netlist Insertion Flow



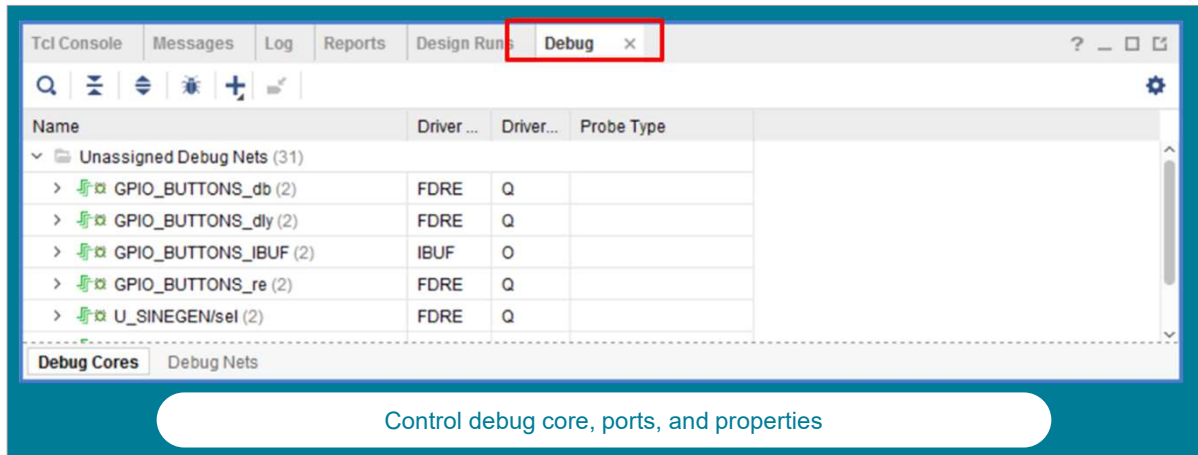
58

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

58

Introduction to the Netlist Insertion Flow



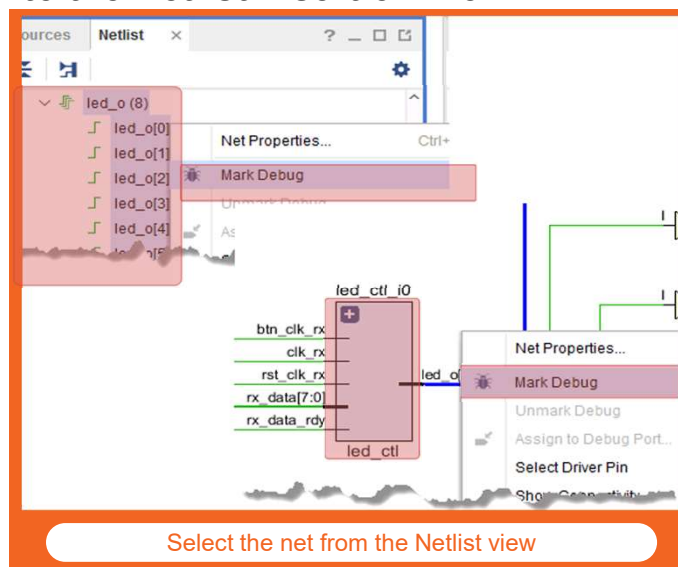
59

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

59

Introduction to the Netlist Insertion Flow



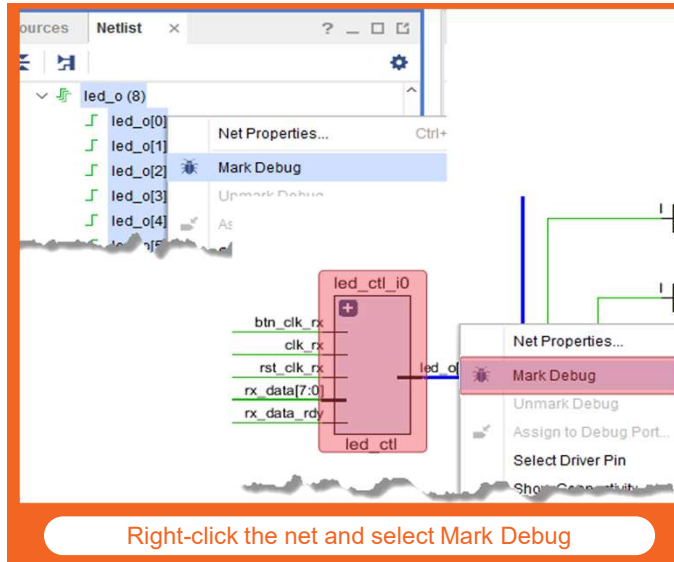
60

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

60

Introduction to the Netlist Insertion Flow



61

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

61

Introduction to the Netlist Insertion Flow

Once the net is marked for debug:

Vivado™ Design Suite adds the net name to a temporary folder, which can be looked at by the Set Up Debug Wizard



Set Up Debug Wizard also enables the use of Unassigned Nets folder

Designer can use this temp folder to select the nets



Find capability allows users to search nets of interest in the dialog box

Netlist insertion flow is the most flexible flow as it allows probing at different design levels
 Only the ILA core can be added using this flow

62

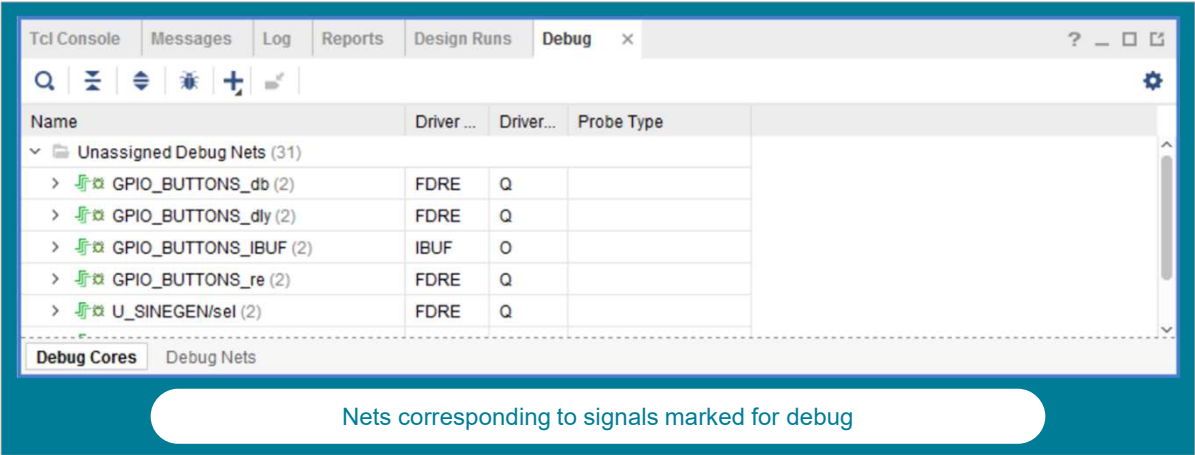
© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

62

Using the MARK_DEBUG Attribute

Identify signals for debugging at the HDL source level prior to synthesis using the MARK_DEBUG constraint



63

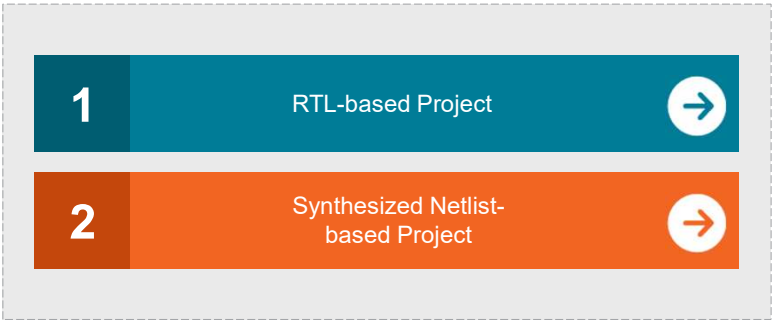
© Copyright 2023 Advanced Micro Devices, Inc.



63

Using the MARK_DEBUG Attribute

Procedure for marking nets for debug depends on two types of projects



64

© Copyright 2023 Advanced Micro Devices, Inc.



64

Using the MARK_DEBUG Attribute

RTL Netlist-based Project

Vivado™ synthesis feature to mark HDL signals for debug using MARK_DEBUG

VHDL

```
attribute MARK_DEBUG : string;
attribute MARK_DEBUG of char_fifo_dout: signal is "true";
```

Verilog

```
(* MARK_DEBUG = "true" *) wire [7:0] char_fifo_dout;
```

- Valid values for the MARK_DEBUG constraint are "TRUE" or "FALSE"
- Vivado synthesis feature does not support the "SOFT" value

65

© Copyright 2023 Advanced Micro Devices, Inc.



 together we advance.

65

Using the MARK_DEBUG Attribute

Synthesized Netlist-based Project

Use the MARK_DEBUG attribute without editing the HDL source files

Add the MARK_DEBUG attribute to the target XDC file:

```
set_property MARK_DEBUG true [get_nets sine*]
```

- Add MARK_DEBUG property on the nets using the synthesized Netlist view

66

© Copyright 2023 Advanced Micro Devices, Inc.



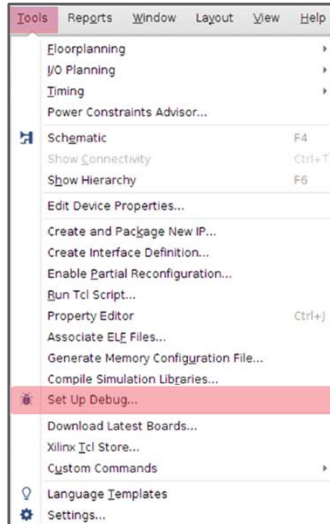
 together we advance.

66

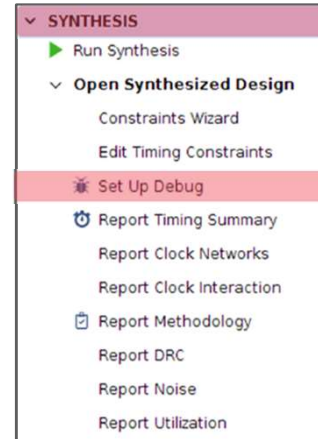
Set Up Debug Wizard to Insert Debug Cores

Once nets are marked for debug, they need to be assigned to debug cores

Set Up Debug Wizard helps in creating debug cores and assigning the debug nets to the inputs of debug cores



OR



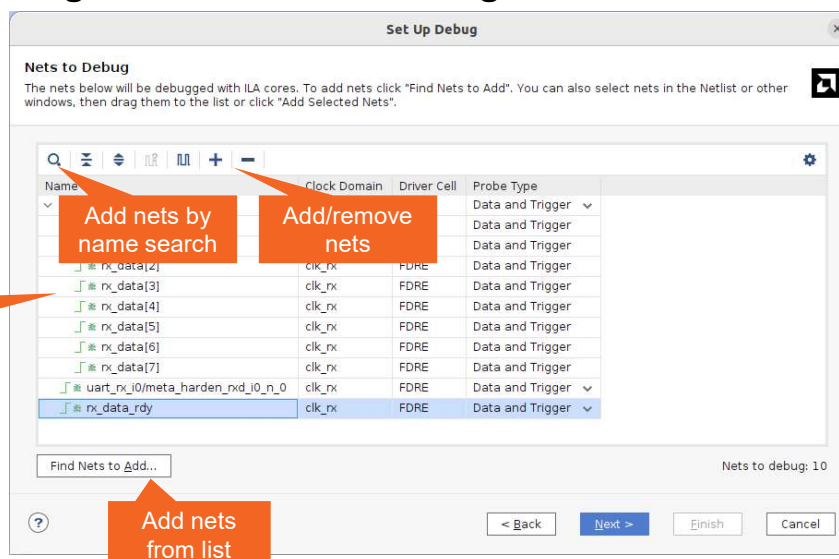
67

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

67

Set Up Debug Wizard to Insert Debug Cores



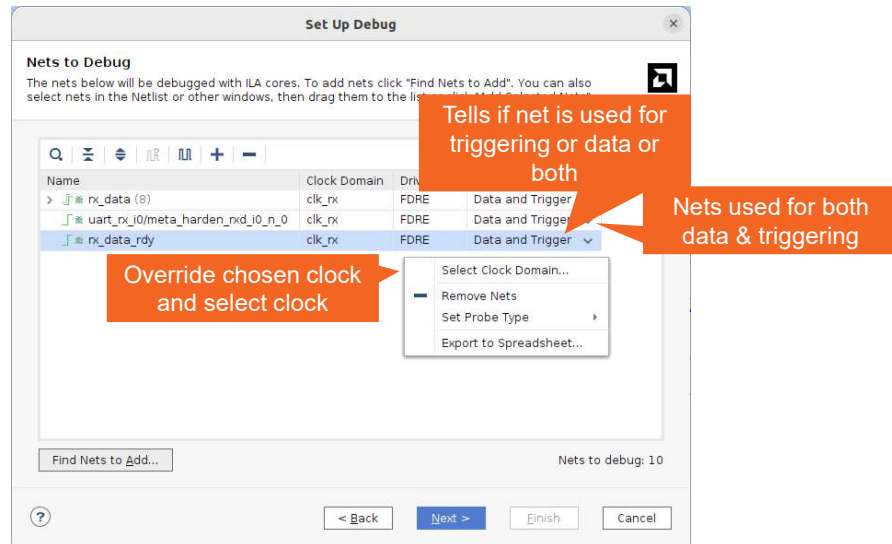
68

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

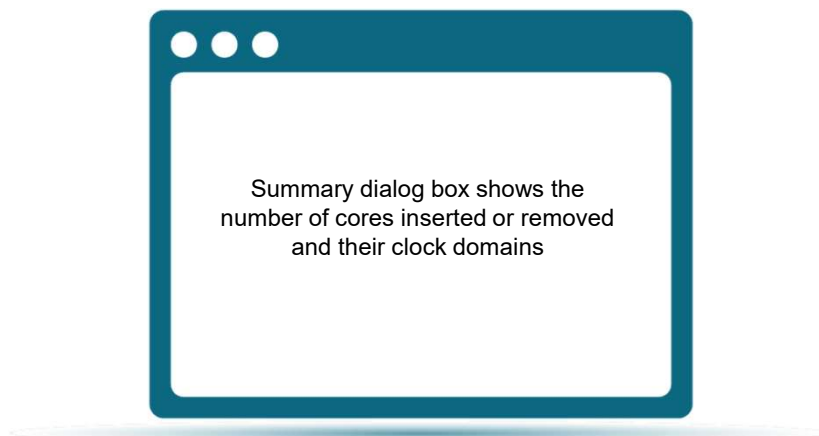
68

Set Up Debug Wizard to Insert Debug Cores



69

Set Up Debug Wizard to Insert Debug Cores

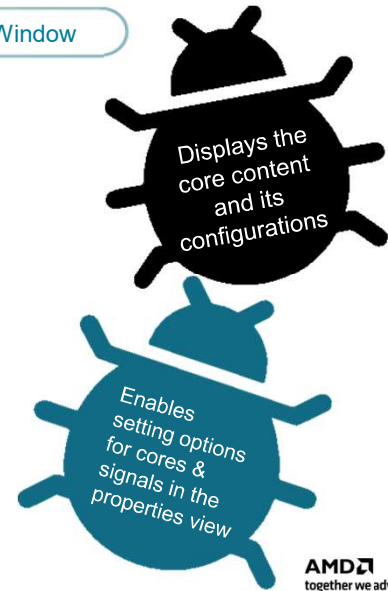
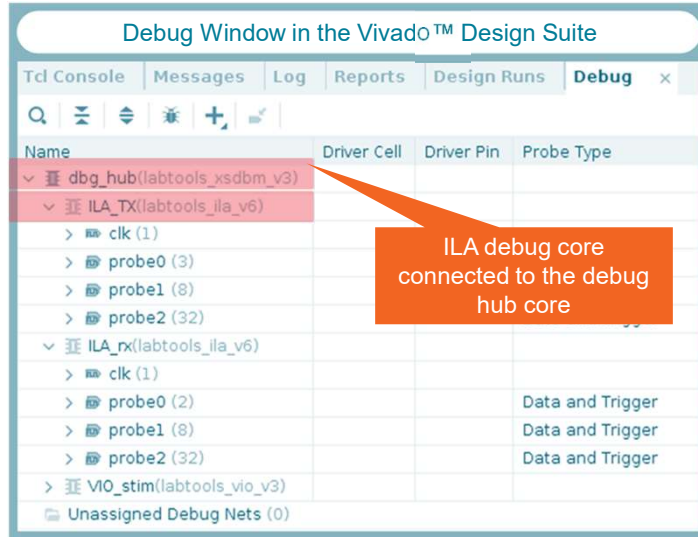


70

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

Debug Core Customization in Debug Window



AMD
together we advance.

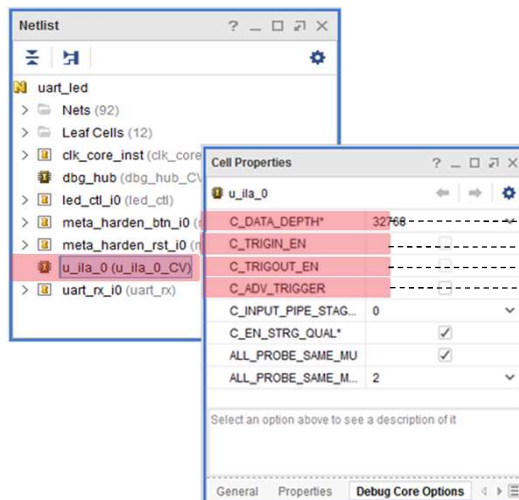
71

© Copyright 2023 Advanced Micro Devices, Inc.

71

Debug Core Customization in Debug Window

To see the Properties view, select debug core u_ila_0 > Debug Core Options in the Instance Properties window



Determines the number of samples to be set up

Enables the TRIG_IN and TRIG_IN_ACK ports

Enables the TRIG_OUT and TRIG_OUT_ACK ports

Enables the advanced triggering option

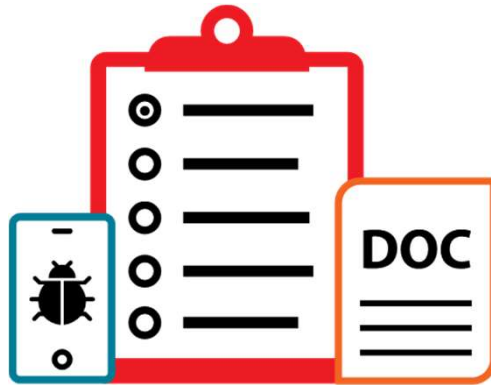
AMD
together we advance.

72

© Copyright 2023 Advanced Micro Devices, Inc.

72

Tcl Commands to Insert Debug Cores



73

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance_

73

Tcl Commands to Insert Debug Cores

- `create_debug_core u_ila_0 labtools_ila_v2`

- Creates an ILA v2.1 core black box
- Creates the CLK port and first probe PROBE0 port of the ILA v2.x core automatically

- `create_debug_port u_ila_0 PROBE`
- `set_property port_width 2 [get_debug_ports u_ila_0/PROBE1]`
- `connect_debug_port u_ila_0/PROBE1 [get_nets [list {A[0]} {A[1]}]]`

- Create more probe ports and set their width to 2 and connect them to the nets to debug

- `set_property C_DATA_DEPTH 1024 [get_debug_cores u_ila_0]`
- `set_property port_width 1 [get_debug_ports u_ila_0/CLK]`
- `set_property port_width 2 [get_debug_ports u_ila_0/PROBE0]`

- Set the data depth of a port to 1024 and set the width of a clock to 1 and width of probe0 to 2

74

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance_

74

Tcl Commands to Insert Debug Cores

- `connect_debug_port u_ila_0/CLK [get_nets [list clk]]`

- Connects the clock port of the ILA to the desired clock net

- `connect_debug_port u_ila_0/PROBE0 [get_nets [list A_or_B]]`

- Connects the probe to the nets to be debugged

75

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance_

75

Implementing Debug Cores in the Vivado IDE

Debug core implementation is done automatically, but can be forced manually for floorplanning/timing analysis



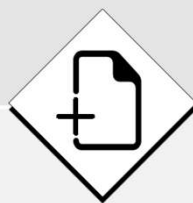
Tcl command for the manual implementation of the debug core

```
implement_debug_core [get_debug_cores]
```

Points to be Considered While Implementing the Debug Core



Vivado™ IDE generates and synthesizes a black-box debug core



Netlists for cores are added to the project



Cores are implemented before the tool launches



Cores are used in all implementation runs



Core connectivity is preserved through netlist updates

After debug core implementation, debug core black boxes are resolved for accessing the generated instances

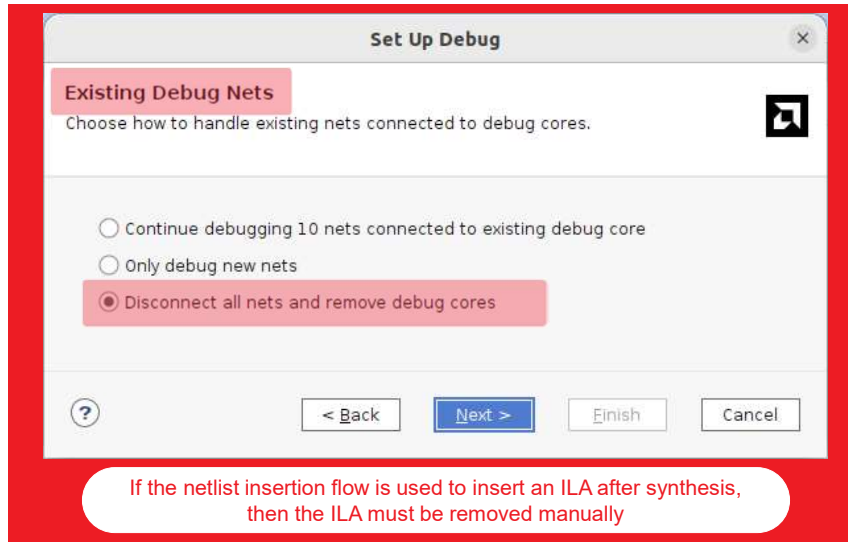
76

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance_

76

Removing Debug Logic after Debug



77

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

77

Configuring the Number of Comparators and Input Pipe Stages

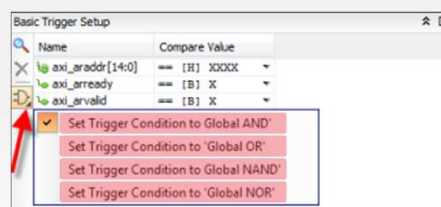
ILA used in the netlist insertion flow allows users to control the number of comparators & the input pipe stage

 **Number of Comparators**

 **Input Pipe Stage**

ILA probe trigger comparators or match units detect specific equality or inequality conditions on the probe inputs

Trigger condition is the result of a Boolean calculation of ILA probe trigger comparator results



78

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
 together we advance...

78

Configuring the Number of Comparators and Input Pipe Stages

ILA used in the netlist insertion flow allows users to control the number of comparators & the input pipe stage



Number of Comparators



Input Pipe Stage

ILA allows setting the number of comparators to be used per probe

To configure more than 64 probe ports use Vivado Tcl Console

General Options Probe_Ports(0..8)

Monitor Type

☒ Native ☐ AXI

Number of Probes 1 [1..1024]

Sample Data Depth 1024

☒ Same Number of Comparators for All Probe Ports

Number of Comparators 1

☐ Trigger Out Port

☐ Trigger In Port

Input Pipe Stages 0

Trigger And Storage Settings

☐ Capture Control

☐ Advanced Trigger

GUI configuration mode is limited to 64 probe ports.

79

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

79

Configuring the Number of Comparators and Input Pipe Stages

ILA used in the netlist insertion flow allows users to control the number of comparators & the input pipe stage



Number of Comparators



Input Pipe Stage

Specify name and options for the new debug core.

Name:

Type:

Options

☒ Rectangular Core

C_DATA_DEPTH 1024

C_TRIGGER_EN ☐

C_TRIGGER_OUT_EN ☐

C_ADV_TRIGGER ☐

C_INPUT_PIPE_STAGES 0

C_EN_STRG_QUAL ☐

ALL_PROBE_SAME_NUM ☒

ALL_PROBE_SAME_MU_CNT 1

Select an option above to see a description of it

OK Cancel

ILA Core Options

Choose features for the ILA debug cores.

Sample of data depth: 1024

Input pipe stages: 0

Trigger and Storage Settings

- ☒ Capture control
- ☒ Advanced trigger

Input Pipe Stage

Allows selection of the number of registers to be added for the probe

Enables extra levels of pipe stages on the PROBE inputs

Improves the timing performance of the design

Takes all valid values from 0 (default) to 6

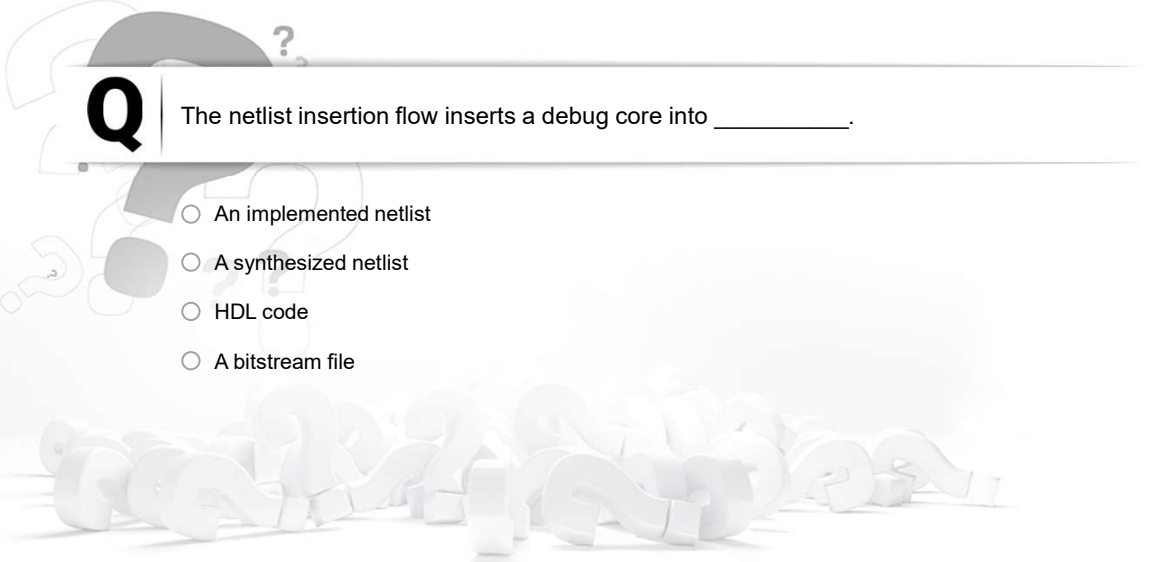
80

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

80

Apply Your Knowledge

**Q**

The netlist insertion flow inserts a debug core into _____.

- ☐ An implemented netlist
- ☐ A synthesized netlist
- ☐ HDL code
- ☐ A bitstream file

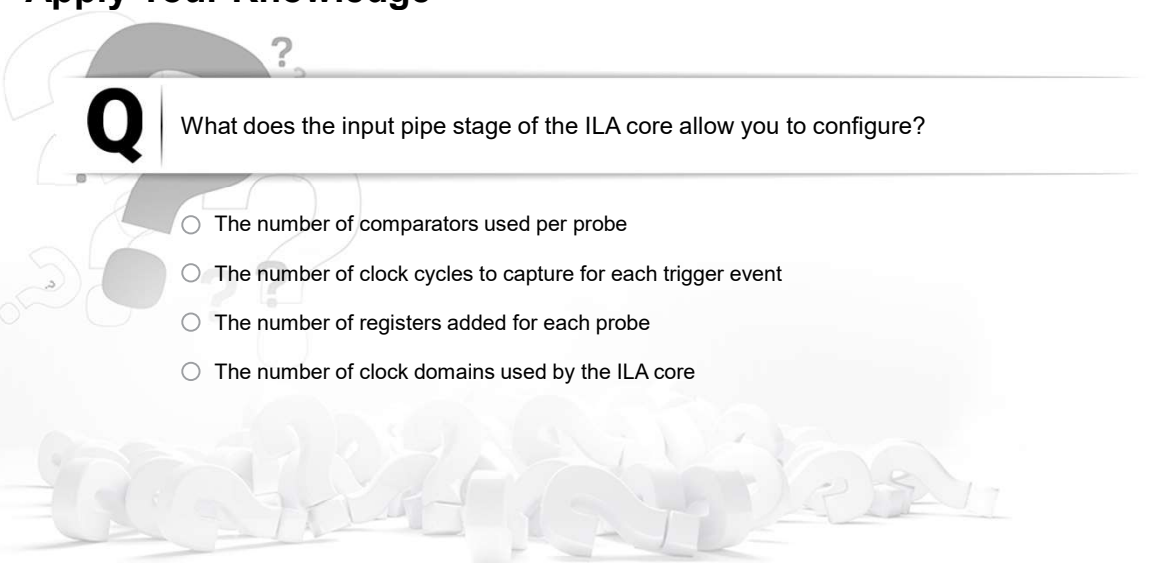
81

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

81

Apply Your Knowledge

**Q**

What does the input pipe stage of the ILA core allow you to configure?

- ☐ The number of comparators used per probe
- ☐ The number of clock cycles to capture for each trigger event
- ☐ The number of registers added for each probe
- ☐ The number of clock domains used by the ILA core

82

© Copyright 2023 Advanced Micro Devices, Inc.

AMD
together we advance.

82

Summary

01 In the netlist insertion flow, the debug cores are added to the synthesized netlist

02 The Set Up Debug Wizard is used to create and configure a debug core

03 The Vivado™ IDE generates and synthesizes a black box debug core

83

© Copyright 2023 Advanced Micro Devices, Inc.



83

GENERAL DISCLOSURE AND ATTRIBUTION STATEMENT

The information contained herein is for informational purposes only and is subject to change without notice. While every precaution has been taken in the preparation of this material, it may contain technical inaccuracies, omissions and typographical errors, and AMD is under no obligation to update or otherwise correct this information. Advanced Micro Devices, Inc. makes no representations or warranties with respect to the accuracy or completeness of the contents of this material, and assumes no liability of any kind, including the implied warranties of noninfringement, merchantability or fitness for particular purposes, with respect to the operation or use of AMD hardware, software or other products described herein. No license, including implied or arising by estoppel, to any intellectual property rights is granted by this document. Terms and limitations applicable to the purchase or use of AMD's products are as set forth in a signed agreement between the parties or in AMD's Standard Terms and Conditions of Sale. GD-18.

©2023 Advanced Micro Devices, Inc. All rights reserved. AMD, the AMD Arrow logo, Vivado, and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective owners.

84

© Copyright 2023 Advanced Micro Devices, Inc.



84



85